COMPUTACION 2021

Clase 5

Si el número de repeticiones

NO

SI

se conoce a-priori

la construcción más apropiada es **WHILE o REPEAT**

Se recomienda **FOR**

(también puede implementarse con while o repeat)

- La característica del ciclo FOR es que se ejecuta una cantidad “conocida” de veces, por lo tanto el control de las repeticiones no depende de evaluar una “condición” (expresión lógica), como el ciclo While o Repeat.
- En el ciclo FOR una sentencia (simple o compuesta) debe ejecutarse repetidamente mientras una progresión (o regresión) de valores es asignada a la *variable de control* de la estructura.
- Se presentan dos casos:

  - ✓ FOR ascendente (FOR-TO)
  - ✓ FOR descendente (FOR-DOWNTO)

Sintaxis de FOR-TO-DO

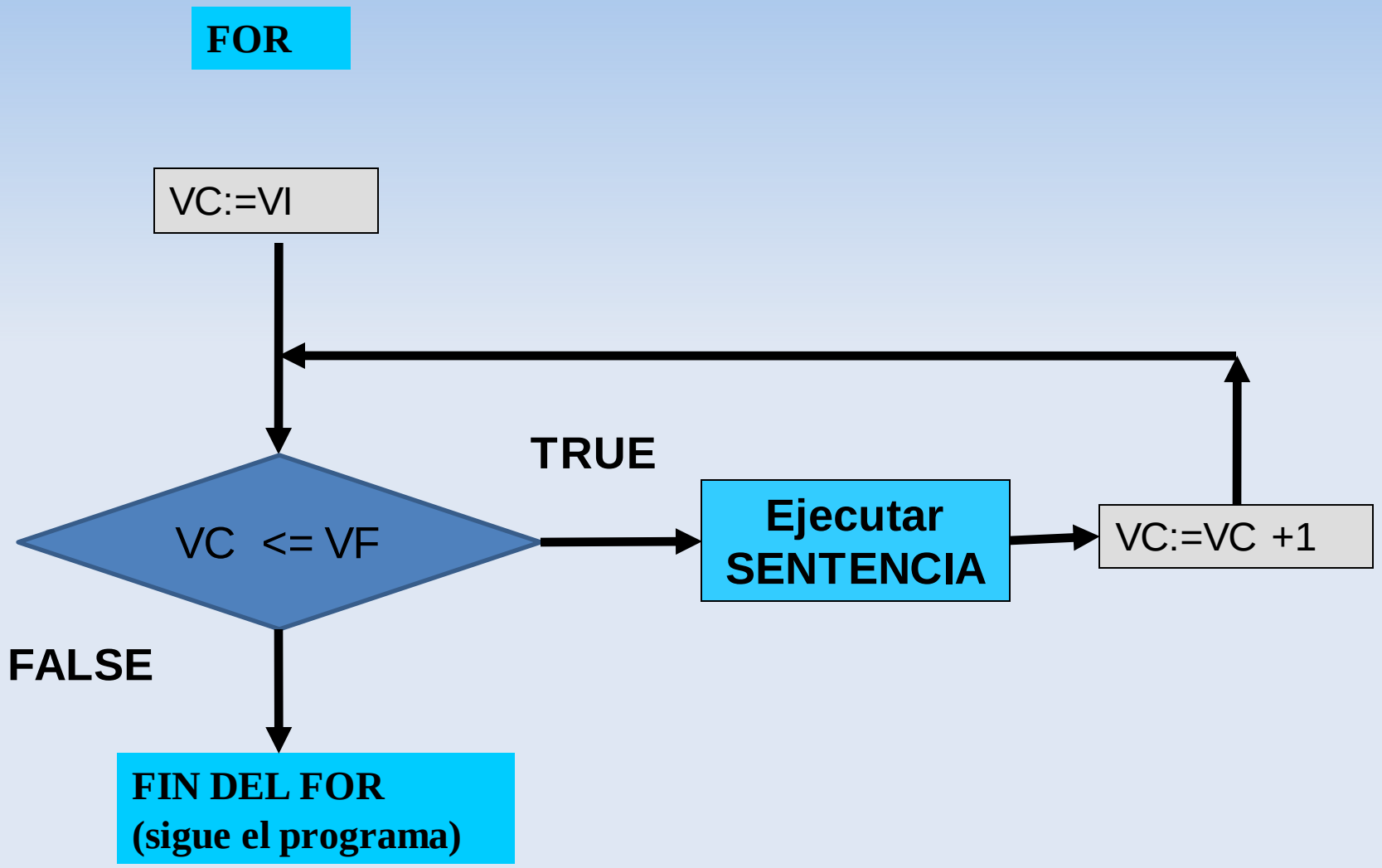
FOR VC:= VI **TO** VF **DO**
Sentencia;

Donde :

VC: es la variable de control.
VI: valor inicial
VF: valor final y VI<=VF

La variable VC toma el valor VI y se incrementa en 1 en cada iteración hasta alcanzar el valor VF.
Previo a cada iteración se verifica VC<= VF.
Estas acciones las realiza la estructura en forma automática.

FOR VC:= VI TO VF DO



E7) Leer un valor K (entero), luego calcular y mostrar los primeros 10 elementos de la tabla de multiplicar de K.

```
Program E7;
 Var
   K,i:integer;
 Begin
  Writeln('Ingrese K');
  Readln(K);
  For i:=1 to 10 do
    writeln( K,' * ', i,' = ', K * i);
 end.
```

Observaciones para FOR VC := VI TO VF DO

- La variable de control (**VC**) debe ser de tipo ordinal (entera o char) <u>NO debe ser alterada</u> dentro del FOR.

- El valor inicial y el valor final (**VI** y **VF**):
  - ✓ Se evalúan una sola vez, al comienzo.
  - ✓ Pueden ser constantes, variables o expresiones del mismo tipo (ordinal) que la variable de control.
  - ✓ No deben ser <u>alterados</u> dentro del FOR.

- Si el <u>valor inicial es mayor que el final no se ejecuta el ciclo.</u>

- El valor de la variable de control queda indefinido luego de terminar el ciclo FOR.

E8) Leer N números reales, seleccionar y mostrar el máximo.
a) Se utiliza una variable para almacenar el máximo que se inicia en valor imposible (menor al rango inferior de los valores a leer)

```
Program E8a;
 Var
   N,i:byte;
   Max, Num: real;
 Begin
 Writeln('Ingrese N');
 Readln(N);
 Max:= -9999;
 for i:=1 to N do
    begin
    readln (Num);
    If Num > Max then
       Max:= Num;
    end;
 writeln ('el maximo es', Max:8:2)
 end.
```

E8 b) Se utiliza una variable para almacenar el máximo que se inicia con el primer valor ingresado.

```
Program E8b;
 Var
   N,i:byte;
   Max, Num: real;
 Begin
 Writeln('Ingrese N');
 Readln(N);
 Readln(Max);
 for i:=2 to N do
      begin
      readln (Num);
      If Num > Max then
           Max:= Num;
      end;
 writeln ('el maximo es', Max:8:2)
 end.
```

8 c) Idem 8 b) y además mostrar el orden en el que ha ingresado el número que resultó máximo

```
Program E8c;
 Var
   N,i, Orden:byte;
   Max, Num: real;
 Begin
 Writeln('Ingrese N');
 Readln(N);
 Readln(Max);
 Orden := 1;
 for i:= 2 to N do
   begin
       readln (Num);
       If Num > Max then
       Begin
             Max:= Num;
             Orden:= i;
       End;
   end;
 writeln ('el maximo es', Max:8:2, 'ingreso en el orden ', Orden)
 end.
```

Sintaxis de FOR-DOWNTO-DO

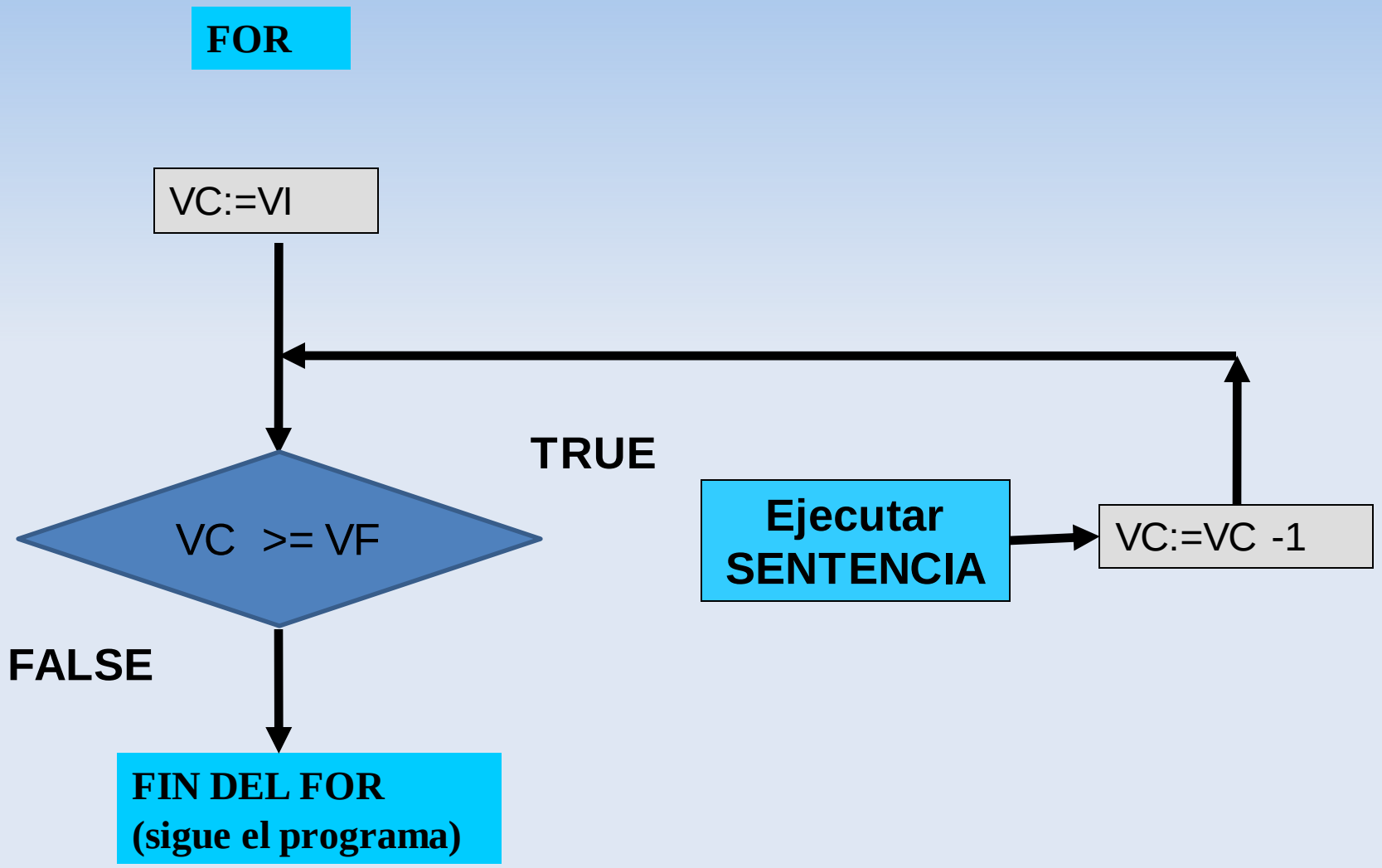
FOR VC:= VI **DOWNTO** VF **DO**
Sentencia;

Donde :

VC: es la variable de control.
VI: valor inicial
VF: valor final y VI>=VF

La variable VC toma el valor VI y se decrementa en 1 en cada iteración hasta alcanzar el valor VF,
Previo a cada iteración se verifica VC>= VF.
Estas acciones las realiza la estructura en forma automática.

FOR VC:= VI DOWNTO VF DO



Cantidad de repeticiones del ciclo FOR

FOR – TO → VF – VI + 1 (incremento 1)

Si el VI>VF el ciclo no se ejecuta ni una vez

FOR – DOWNTO → VI – VF + 1 (decremento 1)

Si el VF>VI el ciclo no se ejecuta ni una vez

```
For i:= 3  to  10 do
    ......................
```

8 veces

```
For i:= 3  to  1 do
    ......................
```

0 veces

```
For i:= -10  to  1 do
    ......................
```

12 veces

```
For c:= 'c'  to  'g' do
    ......................
```

5 veces

```
For i:= 10  downto  3 do
    ......................
```

8 veces

Ejercicio

Para un conjunto de N alumnos, leer su nombre y dos notas. Mostrar para cada alumno su promedio y el nombre del alumno que ha obtenido el mayor promedio (si hubiera más de uno mostrar sólo el primero)

Ciclos Secuenciales

Son independientes, después que finaliza el primer ciclo, comienza el segundo.

```
Program   CiclosSecuenciales;
Var
    Letra: Char;
    Nro:byte;
Begin
For Nro:= 1 to 5 do
     Write (Nro:3);
Writeln;
For Letra:= 'a' to 'z' do
     Write (Letra:3);
End.
```

```
1 2 3 4 5
a b c d e f g h i j k l m n o p q r s t u v w x y z
```

Total de repeticiones

5 + **26**

Ciclos Anidados

Son dependientes, por cada iteración del ciclo externo se ejecuta integramente el ciclo interno.

```
Program   CiclosAnidados;
Var
     Letra: Char;
     Nro:byte;
Begin
For Nro:= 1 to 5 do
     begin
     Write (Nro:3);
     For Letra:= 'a' to 'z' do
           Write (Letra:3);
     Writeln;
     End;
End.
```

```
1 a b c d e f g h i j k l m n o p q r s t u v w x y z
2 a b c d e f g h i j k l m n o p q r s t u v w x y z
3 a b c d e f g h i j k l m n o p q r s t u v w x y z
4 a b c d e f g h i j k l m n o p q r s t u v w x y z
5 a b c d e f g h i j k l m n o p q r s t u v w x y z
```

Total de repeticiones

5 * 26

Eficiencia algorítmica

- Un mismo problema puede ser resuelto con diversos algoritmos.
- Entonces, puede existir uno mejor que otro.
- Por lo tanto, necesitamos un CRITERIO para elegir el mejor algoritmo

Eficiencia algorítmica

- La ejecución de un algoritmo en una computadora consume **recursos.**

- Los recursos primarios son:
  - El espacio de memoria
  - El tiempo de ejecución

Eficiencia algorítmica

*El **espacio de memoria** utilizado es medido en base al número y tipo de variables incluidas, también tamaño de las estructuras de datos usadas (tablas)*

Eficiencia algorítmica

***El tiempo de ejecución** puede medirse calculando el número de acciones elementales que hará el procesador cuando ejecuta el algoritmo.*

Eficiencia algorítmica

- Si bien este resultado difiere de entrada en entrada, se puede determinar matemáticamente la cantidad de tiempo de ejecución necesaria. Es factible estudiar la eficiencia de un algoritmo con entradas de cualquier tamaño.

- Esta medición no depende del tipo de computadora que ejecutara el algoritmo, ni del lenguaje de programación usado.

Eficiencia algorítmica

*Un enfoque para comparar algoritmos que compiten para resolver el mismo problema es elegir aquel que consume **menos** recursos primarios.*

Producto de dos enteros positivos mediante sumas sucesivas

```
PROGRAM suma1;

VAR
  a,b,multi:longint;
  i:longint;
BEGIN
   WRITE('Ingrese dos enteros positivos');
   READLN(a,b);
   multi:=0;
   FOR i:=1 to a DO
           multi:=multi+b;
   WRITELN('El resultado es =',multi);
   Readln
END.
```

INEFICIENTE !! (pues hay otro que lo resuelve usando menos recursos

Si a>b, se ejecuta mas sumas

Producto de dos enteros positivos mediante sumas sucesivas

```
PROGRAM suma2;
VAR
  a,b,multi:longint;
  i:longint;
BEGIN
WRITE('Ingrese dos enteros positivos');
READLN(a,b);
multi:=0;
IF a<b THEN
   FOR i:=1 to a DO
      multi:=multi+b
ELSE
   FOR i:=1 to b DO
      multi:=multi+a;
WRITELN('El resultado es =',multi);
END.
```

MAS EFICIENTE !!

Eficiencia algorítmica

La cantidad de líneas no determina la eficiencia en tiempo de ejecución

Sumar n numeros y multiplicarlos por 100

```
Program multi1;
Var
  n,i,numero,suma:integer;
BEGIN
  suma:=0;
  WRITE('Ingrese un numero entero >0 :');
  READLN(n);
  FOR i:=1 TO n DO
     Begin
     WRITE('Ingrese un numero entero: ');
     READLN(numero);
     numero:=numero*100;
     suma:=suma+numero
     End;
  WRITELN('El resultado es : ',suma);
END.
```

INEFICIENTE !!

Sumar n numeros y multiplicarlos por 100

```
Program multi2;
Var
  n,i,numero,suma:integer;
BEGIN
  suma:=0;
  WRITE('Ingrese un numero entero >0 :');
  READLN(n);
  FOR i:=1 TO n DO
     Begin
     WRITE('Ingrese un numero entero:');
     READLN(numero);
     suma:=suma+numero
     End;
  suma:=suma*100;
  WRITELN('El resultado es : ',suma);
END.
```

MAS EFICIENTE !!

Eficiencia algorítmica

Dos programas con el mismo numero de líneas no siempre son igualmente eficientes en tiempo de ejecución.